zilliz

RetrievalOps

変化に耐える Production Retrievalの設計と運用

Yulan Yan / Founding Solutions Architect, Japan
Zilliz | October 2026

https://www.linkedin.com/in/yulan-yan-0a899686

LinkedIn



Why I am here

研究から実装まで、検索と言語を一貫して見てきた立場から

顔 玉蘭 Yan, Yulan

Founding Solutions Architect, Zilliz

NLP 研究員
NICT・多言語の情報抽出・質問応答

↓

Data & AI プラットフォームの SA
Databricks・RAG・Agent・Vector Search

↓

ベクトル DBに専念
Zilliz・検索基盤そのものを支援

Zilliz

Milvus の開発元

Milvus
世界で最も広く採用されている
オープンソースのベクトルDB

Zilliz Cloud
Milvus のフルマネージド版（商用 SaaS）
AWS・Google Cloud・Azure

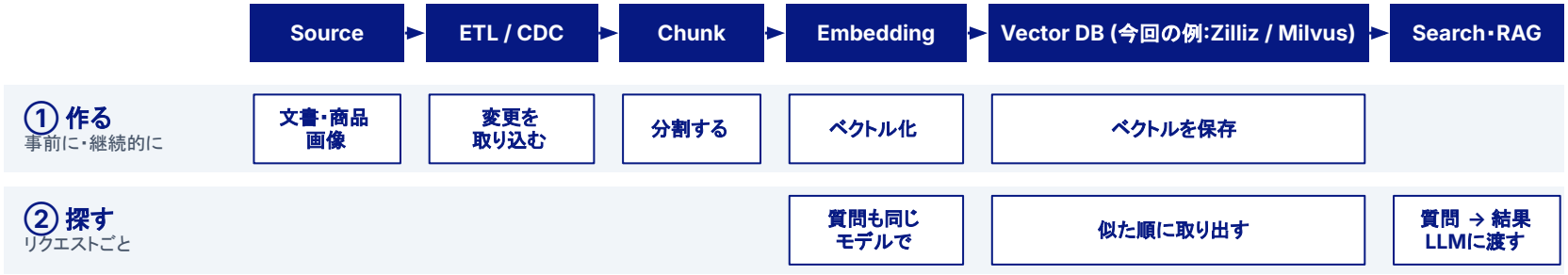
10,000+
の企業が利用
（Zilliz / Milvus）

POINT **今日は、本番に出た検索基盤に「その後」何が起きるかを、事例と一緒にお話しします**

© Copyright 2026 Zilliz

検索基盤の基本:「作る」流れと「探す」流れ

検索・RAG・推薦、どれもだいたいこの形



本番で変わるところ(このあと　1つずつ見ていきます)



POINT **作って終わりではない。本番では、この基盤のあちこちが変わり続ける**

 | © Copyright 2026 Zilliz

zilliz

検索基盤が本番に出た後に、起きること

公開されている顧客事例から、よくある 4つの変化

1 データが変わる
広告が毎日入れ替わる。1日1億件以上のベクトルを更新

SmartNews

3 モデルが変わる
新しいEmbeddingモデルを試すたびに、全件の作り直しに丸 1日

Consensus



2 規模が変わる
データもアクセスも増え、今の構成のままでは遅くなる

Shopee

4 用途が変わる
検索から広告、推薦、RAGへ。同じ基盤に用途が次々に乗る

Cốc Cốc

出典:zilliz.com の顧客事例。会社ごとの事例を、典型的な順に並べたもの

 © Copyright 2026 Zilliz

zilliz

毎日1億件以上が入れ替わっても、広告検索は止めない

1 データが変わる　公開事例：SmartNews

1 データ　2 規模　3 モデル　4 用途

SmartNews

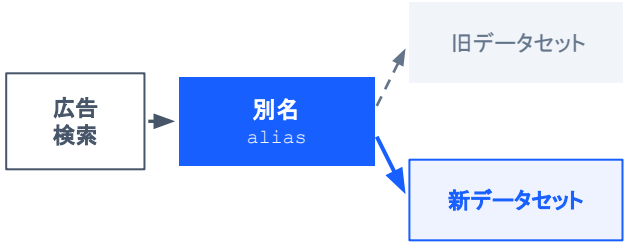
ニュースアプリ（東京）
リアルタイム広告推薦

1億件＋
毎日更新されるベクトル

10ms 未満
広告候補の検索（99%の検索で）

① 広告の候補を探す
ユーザーに近い広告をベクトルで検索

② データの入れ替え
推薦を常に最新のデータで



裏で新しいデータを作り、別名の指す先を付け替える

エンジニアの声： 別名（Collection Alias）で、新旧のデータを「滑らかに」切り替えられる

POINT　大きな入れ替えは「裏で作って、名前を付け替える」。アプリは同じ名前を見続ける

出典：zilliz.com/customers/smartnews

© Copyright 2026 Zilliz

新しいデータ・消すデータ・版をどう扱うか

1 データが変わる　押さえるポイント

1データ　2規模　3モデル　4用途

1 Ground Truthはソース DB、検索側は写し



1件ごとに記録する
- データの版
- 検索側に反映すべき版
- 反映できたか・失敗の回数

定期的に見回って直す
反映漏れ・消し忘れ・版のずれ

2 消すもの・残すものを分ける

ソースで削除	検索側でも削除（削除も連携）
期限切れ	行ごとの有効期限で自動で外す
改定前の版	残す。有効期間で時点を選ぶ
下書き	承認状態で絞り込み、出さない

3 書いた直後、どこまで待つか

最新まで	確実だが遅い
少しだけ	既定。わずかな遅れを許す
自分の分	自分が書いたものは見える
待たない	最速

検索ごとに選べる（consistency level）。
「保存してすぐ確認」する画面だけ「自分の分」に

POINT　検索側はずれる前提。版・状態・消える理由をデータに持たせ、除外は DBに任せる

参考：Milvus Docs「Consistency Level」「Upsert Entities」「Set Collection TTL」

© Copyright 2026 Zilliz

データもアクセスも増え、Shopeeは構成を乗り換えた

2 規模が変わる　公開事例：Shopee

1 データ | **2 規模** | 3 モデル | 4 用途

Shopee

短尺動画サービスの
マルチメディア理解チーム

1億件超
ベクトル

3つ
本番システムが依存

最初の構成	回避策	分散アーキテクチャへ
データとリクエストが増える。 遅くなる	クラスタを複数並べて運用 → 運用コストが大きい。 推薦用は上位の結果をキャッシュし、裏で更新	分散で伸ばせる。複製（Replica）で性能も向上。 キャッシュなしで直接検索

同じ検索基盤に乗る本番システム

動画の推薦	著作権の照合	動画の重複検出

POINT　**規模が変わると、構成を乗り換える日が来る。乗り換えられる設計にしておく**

出典：zilliz.com/customers/shopee

© Copyright 2026 Zilliz

増やすのか、乗り換えるのか

2 規模が変わる 押さえるポイント

1 増やす：何が増えたかで決める

データ量 ↑	計算ユニット（CU）を増やす
アクセス ↑	複製（Replica）を増やす
負荷が波打つ	予測できない波は自動スケール、予測できる波（平日の昼・バッチ）は時間指定
使われないデータ	メモリから外しておく

止めずに、その場で増減できる

2 乗り換える：要件が変わったら

SLA 99.99%・マルチリージョン	→ Business Critical
データを自社のクラウド環境から出せない	→ BYOC
負荷が不定期・小さく始めたい	→ Serverless

止めずに移る（ Zero-Downtime Migration）

新クラスタに全量コピー → 差分を同期 遅れを監視 → 書き込み停止 → 切替（約10秒）

Private Preview。同じか上位のプランへ。旧クラスタは残して確認

POINT 増やすだけなら止めずにその場で。乗り換えるなら新旧を並走させてから切り替える

出典：Zilliz Cloud Docs「Scale Cluster」「Select the Right CU」「Service Plans」「Zero Downtime Migration」

© Copyright 2026 Zilliz

zilliz

毎日作り直して切り替える。新しいモデルもそのまま試せる

3 モデルが変わる　公開事例：Consensus

1 データ　2 規模　**3 モデル**　4 用途

Consensus

研究者向けのAI学術検索
4億件超の学術文献

約1時間
全件の作り直し（以前は24時間超）

+27%
次元を4倍にして検索の質（社内評価）

Collectionを2つ持ち、毎日入れ替える



- 差分は管理しない。毎日、待機側を全件作り直して本番と入れ替える
- 以前は新しいモデルを 1つ試すのに丸 1日。今は同じ作り直しを別のベクトルで回すだけ
- 次元を 4倍（256→1,024）にしても、必要なクラスタは約 2.5倍

POINT　作り直しが速くて安いなら、毎日作り直して切り替えるのが一番シンプル

出典：zilliz.com/customers/consensus（数値は事例に記載の本番値・社内評価）

© Copyright 2026 Zilliz

モデルを替えると、ベクトルの意味ごと変わる

3 モデルが変わる　押さえるポイント

1データ　2規模　**3モデル**　4用途

ベクトルを作る条件

セットで管理し、書き込みと検索で
毎回そろっているか確認する

- モデルと版・次元・正規化
- 文書・質問に付ける定型文
- 文書の切り方
- 類似度の計算方法

**1つでも変われば、
新しく作り直す**

新しいCollectionを裏で作って移行する



① 新しい条件で、別のCollectionを作る

② 過去分を少しずつ投入（途中から再開可）

③ 件数・漏れ・検索結果を確認

④ 一部の検索だけで試す

⑤ 別名の指す先を切り替える

⑥ 古いCollectionはしばらく残す

POINT　モデル・次元・切り方はセットで管理。変えるなら上書きせず、裏で作って切り替える

参考：Zilliz Cloud Docs「Migrate to a New Embedding Model」、Milvus Docs「Manage Aliases」

© Copyright 2026 Zilliz

長い文書は薄まり、断片は上位を埋めてしまう

3 モデルが変わる　長い文書の問題

1 データ　2 規模　**3 モデル**　4 用途

A. 文書全体を1つの単位にすると

→ vector ×1

細かい情報が、文書全体の意味に埋もれる

B. 文書を切った断片（チャンク）を 1行ずつにすると



上位20件のうち6件が同じ報告書の断片

20件の結果が、十数件の資料にしかならない。
アプリ側でまとめ直す処理が必要になる

StructArray：1文書を1行に、チャンクはその中に

報告書 R-031（例）・地域・文書全体のベクトル ■

0	概要　ベクトル ≡
1	地質　ベクトル ≡
2	**鉱化帯の記述**　ベクトル ≡
3	ボーリング結果　ベクトル ≡

結果：報告書 R-031 の3番目（前後のチャンクも一緒に取れる）

- チャンク単位で探して、文書単位で返す
- キーワード検索との併用や、文書ごとのまとめにも対応

POINT　長い文書は、文書とチャンクを 1行にまとめて持つ（StructArray）

参考：Milvus Docs「StructArray」（一部 Preview）。報告書の例は説明用

© Copyright 2026 Zilliz

zilliz

新旧の並べ方は 2つ：Collectionごと、列ごと

3 モデルが変わる　押さえるポイント

1 データ　2 規模　**3 モデル**　4 用途

A. Collection ごと並べる

Consensus



- 裏で作る → 確認 → 別名を切替
- **Consensus**：毎日全件を作り直し（約1時間）、入れ替え

◎ 完全に分離できてシンプル
△ 一時的に容量が2倍。全件を作り直す時間が要る

B. 列ごと並べる

Zilliz / Milvus



◎ 作り直さない。他の列やデータの分け方はそのまま
△ 比較的新しい機能（一部 Preview）

POINT　再構築が速くて安いならA、作り直したくないならB。どちらも旧版は残す

出典：zilliz.com/customers/consensus、Zilliz Cloud Docs「Migrate to a New Embedding Model」。詳細は Appendix

© Copyright 2026 Zilliz

用途は増え続ける：1つの検索から、広告・推薦・ RAGへ

4 用途が変わる　公開事例：Cốc Cốc

1 データ　2 規模　3 モデル　**4 用途**

Cốc Cốc

ベトナムの検索エンジン・ブラウザ
3,000万人以上が利用

約7億
ベクトル（15億まで計画）

5つ
本番システムが同じ基盤に

Zilliz / Milvus
約7億ベクトル

- Web検索の順位付け
- 検索・ショッピング広告
- 関連検索の提案
- 似たユーザーへの広告配信
- RAGチャットボット

以前：ライブラリ（FAISS など）で個別に作っていた
「インデックスであって、システムではない」
今：書き込みやデータ整理は裏で。重い処理は深夜2〜4時に

POINT　検索基盤は最初の1用途で終わらない。複数の用途に共有される前提で作る

出典：zilliz.com/customers/coccoc。5つのシステムは「同じ基盤」の上にあり、同一Collectionとは限らない

© Copyright 2026 Zilliz

同じ基盤を複数の用途で使うときに、決める　3つのこと

4 用途が変わる　押さえるポイント

1 データ | 2 規模 | 3 モデル | **4 用途**

1 見え方：検索ごとに変える

検索画面	意味検索＋キーワード検索
RAG	自部署の文書・本文だけ・上位5件
推薦	文書ごとにまとめて上位50件

絞り込み・返す項目・待ち方は検索ごとに指定できる。アプリからは別名（alias）で参照する

2 負荷：ぶつからないように分ける

複製	読み取りの負荷を分散（Replica）
評価・バッチ	本番のコピー（Snapshot）で回す
時間帯	重い書き込みは夜間に

用途ごとにコピーを作ると、更新・削除の問題がコピーの数だけ増える

3 権限：境界を先に決める

- Cluster
  - Database
    - Collection

権限はこの3階層で設定。行ごとの権限はない

厳密に分ける（法務・人事・顧客）→ DB / Collection を分ける

ゆるく分ける（部署・テナント）→ API側で絞り込みを必ず付ける

POINT　データは1つに。見え方・負荷・権限を用途ごとに分ける

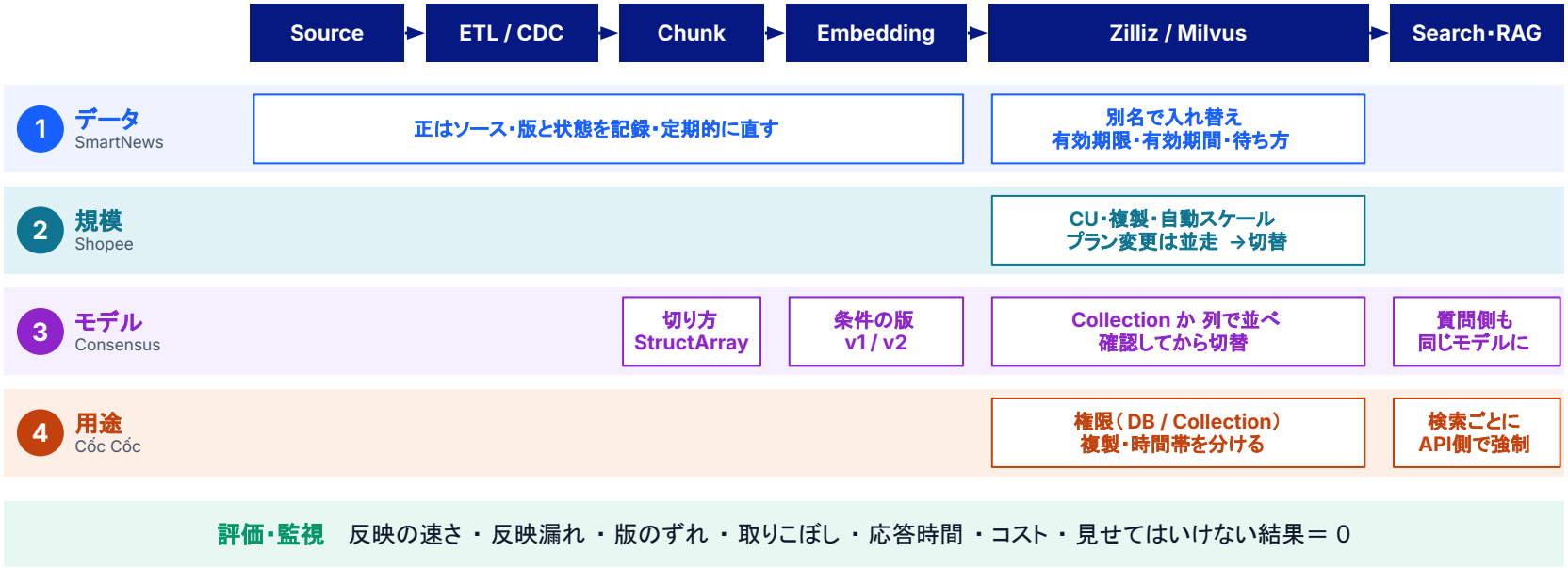
参考：Milvus Docs「Consistency Level」「Manage Aliases」「RBAC」、Zilliz Cloud Docs「Implement Multi-tenancy」

© Copyright 2026 Zilliz

zilliz

4つの変化を1枚に

最初に見たパイプラインの上で、変化ごとに「どこで・何を」するか



 | © Copyright 2026 Zilliz

zilliz

持ち帰ってほしい 4つのこと

1 **データ: Ground Truthはソース、検索側はその写し**
版・状態・消える理由をデータに持たせ、ずれは定期的に直す（SmartNews）

2 **規模：増やすだけならその場で、乗り換えるなら並走**
量とアクセスに合わせて増やす。構成を変えるときは新旧を並べて切り替える（Shopee）

3 **モデル：モデル・次元・切り方はセットで管理**
変えるなら上書きせず、裏で作って確認してから切替。旧版は残す（Consensus）

4 **用途：基盤は 1つ、見え方・負荷・権限は用途ごとに**
最初の1用途で終わらない前提で作る（Cốc Cốc）

初期の検索性能ではなく、変化への追従性を設計する

 © Copyright 2026 Zilliz

zilliz

ありがとうございました

